Sprint 6 – Acceptance Test Planning

1. Define Acceptance Criteria

1.1 Review Original Project and Sprint Requirements

The team reviews all original project documentation, including:

- Product Backlog
- Project requirements (functional & non-functional)
- Sprint 6 Backlog

The goal is to ensure that the Acceptance Criteria align with:

- Flashcard application functionality
- Multilingual requirements (i18n)
- UI/UX requirements
- Code quality expectations after refactoring
- Documentation requirements (ERD, UML, architecture)

1.2 Establish Clear, Measurable Success Criteria

Acceptance criteria are defined across three categories:

Functional Criteria

- The application must load the correct ResourceBundle for the selected language.
- All UI text must display correctly according to the selected language.
- If a key is missing in a ResourceBundle, the application must fall back to the default language (EN).
- No runtime errors should occur after refactoring.
- Flashcard features (create, view, study) must continue to function correctly.

Usability Criteria

- Users must be able to change the language easily from the UI.
- The UI must update immediately after switching languages.
- The system must remember the user's selected language until they change it again.
- No text should be cut off, overflow, or display incorrect fonts after switching languages.
- Font fallback must work correctly for Vietnamese, Finnish, Korean, Lao, etc.

Performance & Reliability Criteria

- ResourceBundle loading must complete in under 50ms.
- The application must not lag when switching languages.
- Fallback must work reliably when keys or bundle files are missing.
- The application must not crash when the user selects a non-existent language.

2. Design Acceptance Tests

2.1 Functional Testing

Tests related to localisation and UI behaviour:

- Verify ResourceBundle loading for each language.
- Verify all UI text updates correctly.
- Verify fallback behaviour when keys are missing.
- Verify Flashcard features after refactoring.

2.2 Usability Testing

- Check the visibility and accessibility of the language selection control.
- Ensure the UI updates instantly after a language change.
- Validate font fallback for special languages.
- Check UI consistency after refactoring.

2.3 Performance & Reliability Testing

- Measure ResourceBundle load time.
- Stress-test language switching (50 iterations).
- Test behaviour when bundle files are missing.
- Test behaviour when keys are missing.

3. Example Test Cases

3.1. TC01 – Load English ResourceBundle (en)

Type: Functional
Precondition: messages_en.properties contains all keys.
Steps:

1. Launch the application (default EN).
2. Open all main screens: Home, Flashcards, Settings.
3. Verify all text.
   Expected:

- All text displays correctly in English.
- No raw keys (e.g., home.title).
- No font issues.
- No warnings in logs.

3.2. TC02 – Load Finnish ResourceBundle (fi)

Type: Functional
Steps:

1. Select Finnish in the UI.
2. Observe the entire UI.
   Expected:

- All text switches to Finnish.
- No English text remains.
- No missing keys.
- Layout remains stable.

3.3. TC03 – Load Korean ResourceBundle (ko)

Type: Functional / Usability
Steps:

1. Select Korean.
2. Check text-heavy screens.
   Expected:

- Korean text displays correctly (no tofu boxes).
- Font fallback works.
- Layout remains intact.

3.4. TC04 – Load Vietnamese ResourceBundle (vi)

Type: Functional / Usability
Steps:

1. Select Vietnamese.
2. Check accented characters.
   Expected:

- All Vietnamese characters display correctly.
- No font issues.
- No clipped text.

3.5. TC05 – Load Lao ResourceBundle (lo)

Type: Functional / Usability
Steps:

1. Select Lao.
2. Check the entire UI.
   Expected:

- Lao text displays correctly.
- Font fallback works.
- No layout overflow.

3.6. TC06 – Load Arabic ResourceBundle (ar) + RTL Layout

Type: Functional / Usability / RTL
Steps:

1. Select Arabic.
2. Check RTL behaviour.
   Expected:

- Arabic text displays correctly.
- If RTL supported:
  - Text right-aligned
  - Layout mirrored appropriately
  - OK/Cancel button positions swapped
- No font issues
- No missing keys

3.7. TC07 – Missing Key Fallback (fi)

Type: Reliability
Precondition: Remove one key from messages_fi.properties.
Steps:

1. Select Finnish.
2. Open the screen containing the missing key.
   Expected:

- Text falls back to English.
- No crash.
- Log shows warning (not error).

3.8. TC08 – Missing Bundle File (ko)

Type: Reliability
Precondition: Delete messages_ko.properties.
Steps:

1. Select Korean.
   Expected:

- Application falls back to EN.
- No crash.
- Log warning.

3.9. TC09 – Stress Test: Switch Languages 50 Times

Type: Performance / Reliability
Steps:

1. Switch languages repeatedly:
   en → fi → ko → vi → lo → ar
2. Repeat 50 times.
   Expected:

- No crash
- No freeze
- No memory leak
- Stable switching time

3.10. TC10 – Performance: Bundle Load Time

Type: Performance
Steps:

1. Measure load time for each language.
   Expected:

- < 50ms
- No UI lag

3.11 TC11 – Key Consistency Across Bundles

Type: Maintainability
Steps:

1. Compare keys across all 6 bundles.
   Expected:

- All bundles contain the same key set.
- Missing keys trigger correct fallback.

3.12. TC12 – UI Consistency After Language Change

Type: Usability
Steps:

1. Select each language.
2. Check UI for leftover text, clipping, and font issues.
   Expected:

- UI is clean and consistent.

3.13. TC13 – RTL Stress Test (Arabic Only)

Type: RTL / Reliability
Steps:

1. Select Arabic.
2. Resize the window repeatedly, switch screens.
   Expected:

- RTL direction preserved
- Layout stable
- No UI breakage

3.14. TC14 – Mixed Content Screen Test

Type: Usability
Steps:

1. Open screen with mixed text types (title, description, buttons, tooltip).
2. Switch through all 6 languages.
   Expected:

- All text updates correctly
- No leftover text
- No font issues

4. Mapping Test Cases to User Stories / Requirements

No.	User Story / Requirement	Test Type	Test Cases
1	Multilingual support (6 languages)	Functional	TC01–TC06
2	Refactoring stability	Functional	TC08
3	Documentation accuracy	Functional	TC10
4	Key consistency	Maintainability	TC11
5	Performance (fast load, smooth switching)	Performance	TC09, TC10
6	Fallback logic	Reliability	TC07, TC08
7	RTL layout stability	RTL / Reliability	TC13
8	UI language switching	Usability	TC05, TC06, TC12
9	Font fallback	Usability	TC03–TC07
10	UI consistency	Usability	TC12
11	Mixed content correctness	Usability	TC14
12	RTL support (Arabic)	Usability / RTL	TC06, TC13